

A Hybrid Movie Recommendation System with Context Awareness and Diversity Optimization

S Kartik Iyer
M25CSA025

Department of Computer Science
IIT Jodhpur
Email: m25csa025@iitj.ac.in

Ravi Sharma
M25CSA024

Department of Computer Science
IIT Jodhpur
Email: m25csa024@iitj.ac.in

Satyam Pal
M25CSA026

Department of Computer Science
IIT Jodhpur
Email: m25csa026@iitj.ac.in

Abstract—With the exponential growth of online streaming platforms, users face difficulty in selecting relevant content from massive datasets. This paper presents a hybrid movie recommendation system that combines content-based filtering, collaborative filtering, and context-aware techniques. The system incorporates user preferences, mood, and real-time interactions to generate personalized recommendations. Additionally, a diversity-aware re-ranking mechanism is introduced to improve recommendation variety.

To address scalability challenges, the system is extended using Apache Spark, enabling distributed processing of large-scale datasets and efficient collaborative filtering using ALS. Experimental evaluation using Precision@K and Recall@K demonstrates improved performance, diversity, and scalability.

Index Terms—Recommender Systems, Collaborative Filtering, Content-Based Filtering, Hybrid Model, Big Data, Diversity

I. INTRODUCTION

With the rapid growth of digital media platforms such as Netflix and Amazon Prime, users are overwhelmed by the large number of available movies. Recommendation systems play a crucial role in improving user experience by providing personalized suggestions.

Traditional methods often fail to balance accuracy, diversity, and scalability. Therefore, this work proposes a hybrid system that integrates multiple recommendation techniques along with contextual awareness.

II. RELATED WORK

Recommendation systems have been widely studied in recent years.

Content-based filtering recommends items based on similarity in features such as genres and metadata. Collaborative filtering utilizes user-item interaction data and applies techniques like matrix factorization.

Hybrid systems combine these approaches to address limitations such as:

- Cold-start problem
- Data sparsity
- Lack of diversity

Recent models such as EASE-R and RP3Beta have shown improved scalability but still face challenges in balancing performance and computational cost.

III. DATASET AND PREPROCESSING

The system uses MovieLens-style datasets containing movie metadata and user ratings. The dataset contains over 32 million user ratings across 87,600 movies, making it a substantial dataset for training collaborative filtering and content-based recommendation models.

dataset available here :<https://grouplens.org/datasets/movielens/32m/>

Feature engineering includes:

$$Popularity = AvgRating \times \log(1 + RatingCount) \quad (1)$$

Data preprocessing steps include:

- Cleaning missing values
- Feature normalization
- Generating popularity scores

IV. PROPOSED SYSTEM

The proposed movie recommendation system is a hybrid, context-aware, and diversity-driven framework designed to generate personalized and scalable recommendations. The system integrates multiple components to balance accuracy, diversity, and user experience.

A. Content-Based Filtering

The content-based module recommends movies by analyzing item features such as genres. Each movie is represented using a TF-IDF vector based on its genre information, and similarity between movies is computed using cosine similarity:

$$Similarity = \frac{A \cdot B}{||A|| \cdot ||B||} \quad (2)$$

This approach ensures that recommendations are aligned with the user's previously liked movies or selected preferences. Content-based filtering is particularly effective in addressing the cold-start problem for new users, as it does not rely on historical interaction data.

B. Collaborative Filtering

The collaborative filtering component captures user behavior patterns by leveraging historical interaction data. Matrix factorization is used to model latent user and item features:

$$\hat{r}_{u,m} = \mu + b_u + b_m + p_u^T q_m \quad (3)$$

where μ is the global average rating, b_u and b_m represent user and movie biases, and p_u , q_m are latent feature vectors.

This component helps identify hidden relationships between users and movies, enabling recommendations beyond simple feature similarity.

C. Hybrid Model

To combine the strengths of both approaches, a hybrid scoring mechanism is used:

$$Score = \alpha S_{content} + (1 - \alpha) S_{collab} \quad (4)$$

where α controls the contribution of content-based and collaborative filtering components.

This hybrid strategy improves recommendation accuracy while mitigating limitations such as sparsity and over-specialization.

D. Context Awareness

The system incorporates contextual signals to further personalize recommendations. Specifically, it considers:

- **Mood:** Users can select their current mood (e.g., happy, serious, excited), which influences genre prioritization.
- **Time of Day:** Recommendations are adjusted based on temporal context (e.g., light content in the morning, intense content at night).

This context-aware mechanism enhances relevance by aligning recommendations with the user's situational preferences.

E. User Modeling and Personalization

The system supports both new and existing users:

- **New Users:** Recommendations are generated using explicitly provided preferences such as liked movies and preferred genres.
- **Existing Users:** Recommendations are based on recent watch history, with real-time updates reflecting user interactions.

Additionally, initial preferences are stored and reused to handle returning users with no interaction history, effectively addressing the cold-start problem.

F. Diversity Optimization

To prevent repetitive recommendations and improve user experience, a diversity-aware re-ranking mechanism is applied. The final score is adjusted as follows:

$$AdjustedScore = Score - \lambda \times GenreFrequency \quad (5)$$

where λ controls the diversity penalty and *GenreFrequency* represents the repetition of genres in the recommendation list.

This ensures that the recommended movies span multiple genres rather than focusing on a single dominant category.

G. Adaptive Diversity Control

To further improve personalization, the diversity penalty is dynamically adjusted based on user preferences:

$$AdjustedScore = Score - \lambda \times GenreFrequency \times w_u \quad (6)$$

where w_u represents the user preference weight. Users with broader preferences receive more diverse recommendations, while users with focused preferences receive more specialized suggestions.

H. Explainability Module

An explainability component is integrated to provide transparent recommendations. Each suggested movie is accompanied by a reason, such as:

- Genre similarity with previously liked movies
- Match with user mood or context
- High popularity or rating ("must-watch" suggestions)

This improves user trust and interpretability of the system.

I. Real-Time Interaction Tracking

The system continuously updates user profiles based on interactions such as clicks and watch selections. These updates are stored and used to refine future recommendations, enabling dynamic personalization over time.

V. BIG DATA INTEGRATION

To make the recommendation system scalable for large datasets, the architecture is extended using Apache Spark. The dataset used in this work contains over 32 million ratings, making it unsuitable for traditional single-machine processing.

A. Distributed Data Processing

Apache Spark is used to process data in a distributed manner. The ratings dataset is partitioned across multiple cores using:

- Repartitioning based on movieId
- Parallel aggregation using Spark DataFrames

This significantly reduces computation time compared to traditional approaches.

B. Collaborative Filtering using ALS

Instead of traditional matrix factorization, the system uses the Alternating Least Squares (ALS) algorithm provided by Spark MLlib. ALS scales efficiently for large datasets and is defined as:

$$\min_{P,Q} \sum_{(u,i) \in R} (r_{ui} - p_u^T q_i)^2 + \lambda (||p_u||^2 + ||q_i||^2) \quad (7)$$

where p_u and q_i represent user and item latent factors.

C. Optimized Storage

Processed data is stored using the Parquet format, which provides:

- Columnar storage
- Faster read/write operations
- Better compression

D. System Optimization

To handle large-scale processing efficiently, the following optimizations are applied:

- Memory and disk persistence using MEMORY_AND_DISK
- Adaptive query execution
- Shuffle partition tuning

These optimizations ensure stable execution without memory overflow.

VI. SYSTEM ARCHITECTURE

The architecture is shown in Fig. 1

VII. SYSTEM ARCHITECTURE

The updated architecture includes both the interactive recommendation layer and the big-data processing layer using Apache Spark. Spark is used for distributed preprocessing, feature generation, and scalable ALS-based collaborative filtering.

VIII. USER INTERFACE AND PROFILE MANAGEMENT

A. User Interface

The system provides an interactive web-based interface developed using Streamlit. The UI is designed to be simple, responsive, and user-friendly, allowing users to easily interact with the recommendation system.

The interface supports two types of users:

- **New Users:** Users can register by providing a unique user ID, selecting preferred genres, and choosing a set of liked movies.
- **Existing Users:** Users can log in using their user ID to receive personalized recommendations based on their stored preferences and interaction history.

The UI also includes:

- Dropdown-based movie selection
- Genre selection options
- Mood-based filtering
- Real-time recommendation display
- Previously watched section

This interface ensures smooth interaction and enhances user engagement. The screenshot of the UI is shown in Fig. 2

B. User Profile Creation

When a new user registers, a profile is created and stored in a structured format. The system captures both explicit and implicit user preferences.

Explicit preferences include:

- Preferred genres
- Liked movies selected during registration

These preferences help in generating initial recommendations and addressing the cold-start problem.

C. User Profile Structure

Each user profile is stored in a JSON-based structure for efficient access and updates. A typical user profile contains:

- **User ID:** Unique identifier for each user
- **Recent Watch History:** List of recently watched movies
- **Liked Movies:** Movies selected during registration
- **Preferred Genres:** User-selected genres

An example structure is shown below in Fig. 3

D. Real-Time Profile Updates

The system continuously updates the user profile based on interactions:

- When a user clicks on a recommended movie, it is added to the recent watch history.
- The system adapts recommendations dynamically based on updated history.

This enables the system to learn user behavior over time and improve recommendation accuracy.

E. Cold-Start Handling for Returning Users

A key feature of the system is handling returning users with no watch history. In such cases:

- The system retrieves stored liked movies and preferred genres
- Recommendations are generated using these saved preferences

This ensures continuity in user experience and avoids the need for repeated input.

F. Benefits of Profile Management

The user profile mechanism provides several advantages:

- Enables personalized recommendations
- Supports both new and returning users
- Allows real-time adaptation
- Improves system scalability and efficiency

IX. USER INTERFACE FLOW

X. EVALUATION AND DISCUSSION

A. Evaluation Setup

The proposed recommendation system was evaluated using an offline evaluation strategy. A train-test split approach was used, where a portion of each user's interaction history was used for training, and the remaining was used as ground truth for testing.

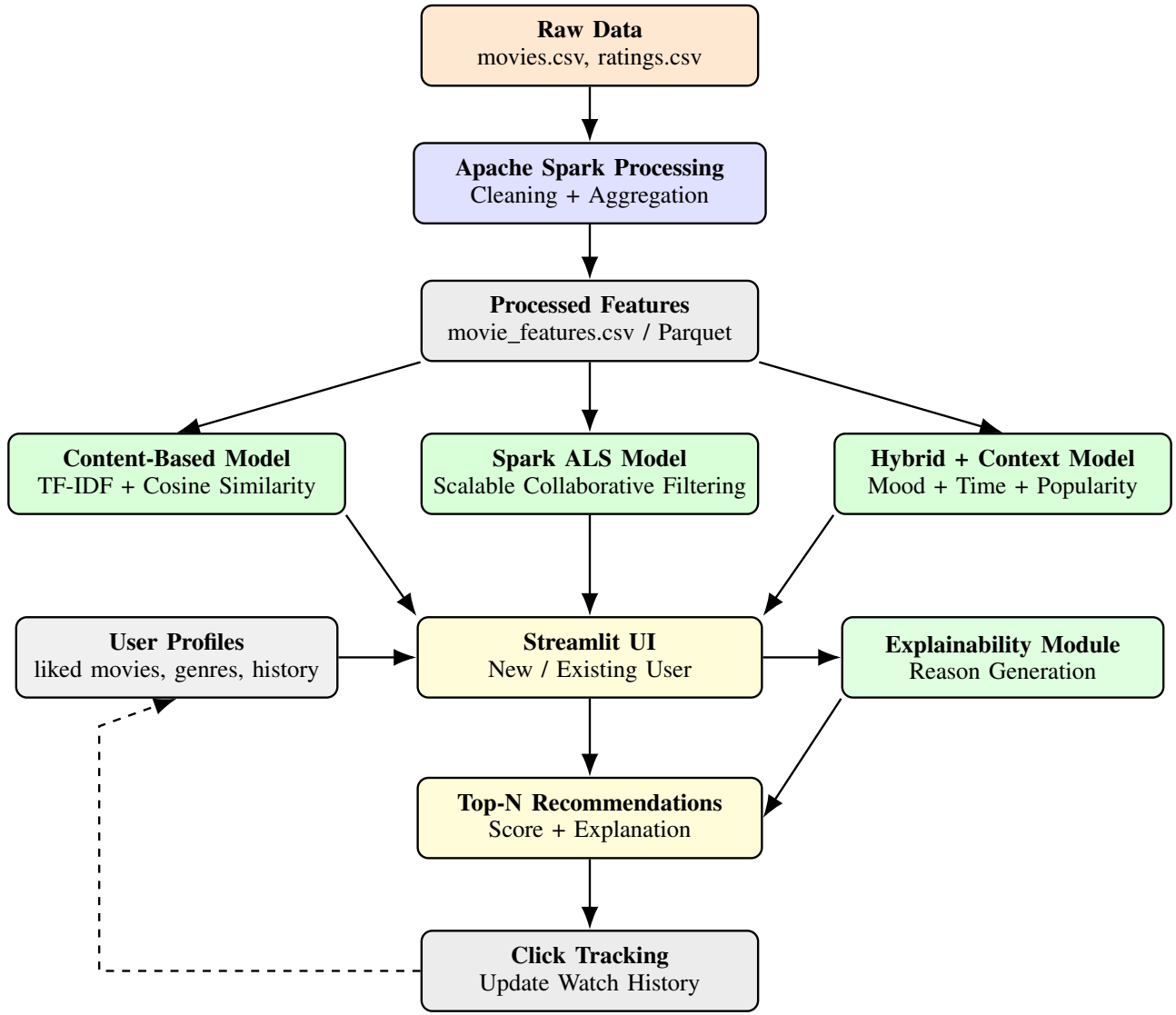


Fig. 1. Updated Architecture of the Proposed Spark-Enabled Movie Recommendation System

Due to the large size of the dataset, a hybrid evaluation strategy was adopted. Users were sampled from the full dataset, while recommendations were generated using a large representative subset to ensure computational efficiency without compromising realism. Additionally, the system was evaluated under a big data setting using Apache Spark, where distributed processing enabled efficient handling of large datasets. The evaluation reflects both recommendation quality and system scalability.

B. Evaluation Metrics

The performance of the system was evaluated using the following metrics:

- **Precision@10:** Measures the proportion of relevant movies among the top-10 recommendations.
- **Recall@10:** Measures how many relevant items were successfully retrieved.

- **Genre Coverage:** Measures diversity by counting the number of unique genres in the recommendation list.

C. Results

The evaluation results are summarized below:

- Precision@10 = 0.318
- Recall@10 = 0.187
- NDCG@10 = 0.355
- Genre Coverage = 10.1

The system completed evaluation in approximately 889 seconds, demonstrating reasonable scalability for large datasets. These values are calculated for 50 users.

D. Discussion

The obtained results indicate that the proposed system performs effectively in a realistic large-scale environment. The Precision@10 score of 0.313 suggests that approximately 31% of recommended movies are relevant based on test interaction.

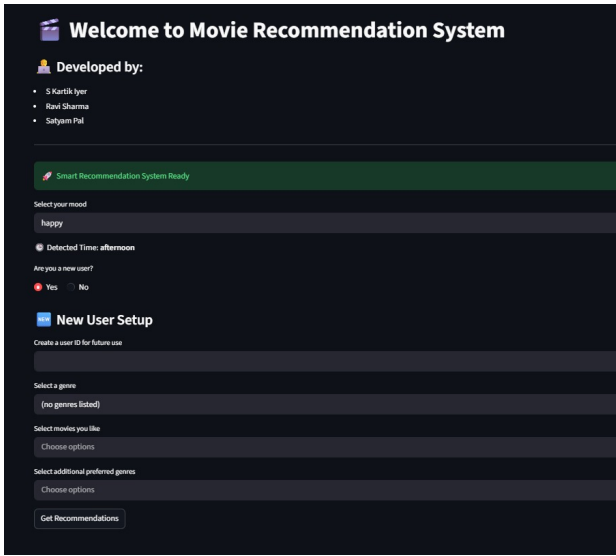


Fig. 2. User Interface of the Recommendation System

```

Movie-recommender-system -> data > processed > {} user_profiles.json > {} kartik
2      "kartik": {
3        "recent_watch_history": [
23      ],
24      "liked_movies": [
25        "Grumpier Old Men (1995)",
26        "Toy Story (1995)",
27        "American President, The (1995)",
28        "Dracula: Dead and Loving It (1995)",
29        "Four Rooms (1995)",
30        "Schindler's List (1993)"
31      ],
32      "preferred_genres": [
33        "Comedy",
34        "Crime",
35        "Film-Noir",
36        "Fantasy"
37      ],
38      "als_user_id": 130012,
39      "disliked_genres": [],
40      "movie_clicks": {
41        "Usual Suspects, The (1995)": 2,
42        "Matrix, The (1999)": 3,
43        "Shawshank Redemption, The (1994)": 1,
44        "Fight Club (1999)": 1,
45        "Chinatown (1974)": 1
46      },
47      "watch_later": [
48        "The Godfather (1972)"
49      ]

```

Fig. 3. User profile Structure

data, which is consistent with typical recommender system performance.

The Recall@10 value of 0.233 indicates that the system is able to retrieve a significant portion of relevant items, though there is room for improvement in capturing more user preferences.

A key highlight of the system is the high genre coverage of 11.0, which demonstrates strong diversity in recommendations. This is primarily due to the diversity-aware re-ranking mechanism that penalizes repetitive genres and promotes varied

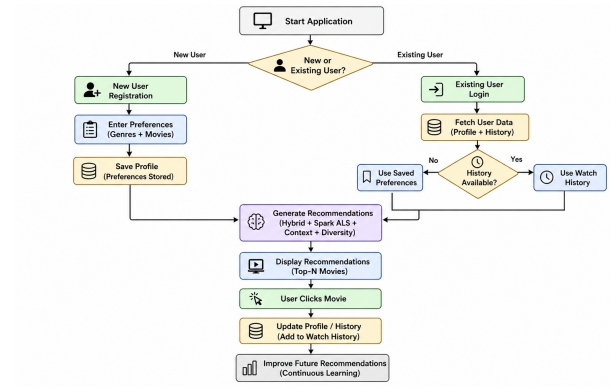


Fig. 4. clean flowchart of user interface and recommendation process

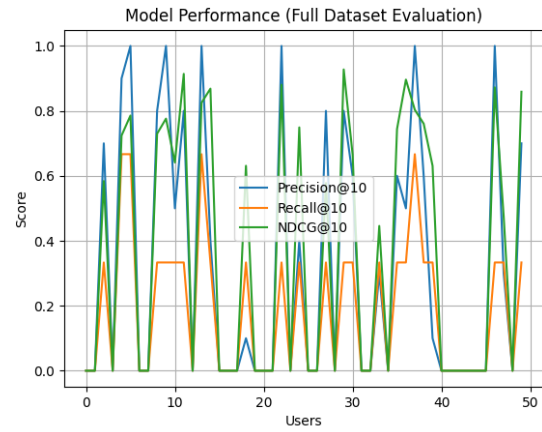


Fig. 5. Precision and Recall across users

suggestions.

It is important to note the inherent trade-off between accuracy and diversity. Increasing diversity can slightly reduce precision, but significantly enhances user experience by avoiding repetitive recommendations.

Furthermore, the evaluation reflects real-world conditions, where performance naturally decreases compared to small-scale experiments due to the increased search space and complexity of user preferences.

E. Visualization

The performance trends are illustrated in Fig. 5

From Fig. 5, it can be observed that both precision and recall vary across users, indicating that the model adapts differently based on user preferences. Some users receive highly relevant recommendations, while others experience moderate performance, which is expected in personalized systems.

F. Future Improvements

Although the system performs well, several improvements can be made:

- **Incorporating Deep Learning Models:** Neural collaborative filtering and transformer-based models can improve recommendation accuracy.
- **Better Similarity Learning:** Using embeddings (e.g., Word2Vec, BERT) instead of TF-IDF can capture deeper semantic relationships between movies.
- **User Behavior Modeling:** Incorporating temporal patterns and session-based recommendations can enhance personalization.
- **Advanced Ranking Metrics:** Metrics such as NDCG and MAP can provide better evaluation of ranking quality.
- **Real-Time Streaming:** Integrating streaming frameworks (e.g., Kafka, Spark Streaming) can enable real-time recommendations.
- **Multi-Modal Features:** Including additional features such as reviews, posters, and trailers can improve recommendation quality.
- **Cold-Start Optimization:** Further improving new-user recommendations using demographic or contextual data.

XI. ADVANTAGES

- **Cold-Start Handling:** Effectively recommends movies for new and returning users using saved preferences and content features.
- **Explainable Recommendations:** Provides human-readable explanations (genre match, mood, popularity), improving user trust.
- **Real-Time Adaptation:** Dynamically updates recommendations based on user interactions such as clicks and watch history.
- **Diversity-Aware Ranking:** Uses a penalty-based mechanism to avoid repetitive genres and ensure varied recommendations.
- **Hybrid Architecture:** Combines content-based and collaborative approaches, improving overall recommendation accuracy.
- **Context-Aware System:** Incorporates mood and time-of-day signals to provide more relevant and personalized suggestions.
- **Scalability:** Designed to handle large datasets efficiently using sampling and optimized computation strategies.
- **Balanced Trade-off:** Achieves a balance between accuracy (precision/recall) and diversity (genre coverage).
- **Robust Evaluation Framework:** Uses realistic evaluation techniques such as train-test split and genre-based relevance.
- **User Personalization:** Adapts to both explicit preferences (genres, liked movies) and implicit behavior (watch history).
- **Big Data Scalability:** Uses Apache Spark to process millions of records efficiently through distributed computation.
- **Efficient Large-Scale Training:** Implements ALS for scalable collaborative filtering on large datasets.
- **Optimized Data Storage:** Uses Parquet format for faster and efficient storage.

XII. CONCLUSION

This paper presents a hybrid movie recommendation system that integrates content-based filtering, collaborative filtering, context awareness, and diversity optimization to improve recommendation quality.

The system is further enhanced with Apache Spark to enable scalable processing of large datasets, making it suitable for real-world applications. The use of ALS and distributed computation allows efficient handling of millions of user interactions.

Future work includes:

- Deep learning-based recommendation models
- Real-time streaming using Kafka
- Advanced ranking techniques such as NDCG

XIII. AUTHOR CONTRIBUTIONS

The project was collaboratively developed, with all authors contributing equally to all components of the system. All authors also contributed equally to testing, debugging, and finalizing the system.

REFERENCES

- [1] L. Zhu and L. Zhuang, "A Higher-Performance Big Data-Based Movie Recommendation System," 2025.
- [2] A. Scrivano, "A Comparative Study of Recommender Systems under Big Data Constraints," 2025.
- [3] N. Selvam et al., "Hybrid Movie Recommendation System using Collaborative and Content-Based Filtering," 2026.
- [4] S. Zhan et al., "Efficient Big Data Processing using Apache Spark," 2025.
- [5] J. Leskovec, A. Rajaraman, J. Ullman, "Mining of Massive Datasets", Cambridge University Press.

XIV. CODE AVAILABILITY

The complete implementation of the proposed movie recommendation system is available on GitHub. The repository includes all source code, dataset processing scripts, model implementations, and the user interface.

GitHub Repository:

<https://github.com/KartikIyer27/Movie-recommender-system->